
Chapter 7 Relational Database Design



Relational Database Design

- ▶ In this chapter, we consider the problem of **designing a schema for a relational database**.
- ▶ In general, the goal of relational database design is to generate a set of relation schemas that allows us to *store information without unnecessary redundancy*, yet also allows us to *retrieve information easily*.
- ▶ This is accomplished by designing schemas that are in an appropriate **normal form**.



Outline

- ▶ **Features of Good Relational Design**
- ▶ **Decomposition Using Functional Dependencies**
- ▶ **Normal Forms (First Form, BCNF, 3NF)**
- ▶ **Functional Dependency Theory**
- ▶ **Algorithms for Functional Dependencies**
- ▶ **Decomposition Using Multivalued Dependencies**
- ▶ **More Normal Form**
- ▶ **Database-Design Process**



Design Alternative: Larger Schemas

- ▶ Suppose we combine *instructor* and *department* into *inst_dept*
 - ▶ *instructor*(ID, name, dept_name, salary),
 - ▶ *department*(dept_name, building, budget)
 - ▶ *inst_dept*(ID, name, salary, dept_name, building, budget)
- ▶ Result is possible **repetition of information**.

ID	name	salary	dept_name	building	budget
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000



Design Alternative: Larger Schemas

► Three Problems

- **Redundancies**: Consider the instance of the *inst_dept* relation, we have to repeat the *department* information (“*building*” and “*budget*”) once for each *instructor* in the *department*.

ID	name	salary	dept_name	building	budget
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000



Design Alternative: Larger Schemas

► Three Problems

- It is important that all these tuples agree as to the *budget* amount since otherwise our database would be **inconsistent**. (considering *some user might update the budget amount in one tuple but not all*)

ID	name	salary	dept_name	building	budget
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000



Design Alternative: Larger Schemas

► Three Problems

- Suppose we are **creating a new department**, we cannot directly create a tuple, unless the department has at least one instructor or create a tuple with a *null* value of *building* and *budget*.

ID	name	salary	dept_name	building	budget
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000



Design Alternative: Smaller Schemas

- ▶ Suppose we had started out with *inst_dept*.
- ▶ **How would we know it should be split into *instructor* and *department*?**
 - ▶ Discovering **repetition** would be costly in a real-world database.
 - ▶ It does not allow us to determine whether the lack of repetition is just *a “lucky” special case* or whether it is *a manifestation of a general rule*.



Design Alternative: Smaller Schemas

- ▶ Write a rule “*if there were a schema (dept_name, budget), then dept_name is able to serve as the primary key.*”
- ▶ Denoted as a **functional dependency**:
$$dept_name \rightarrow budget$$
- ▶ Given such a rule, we now have sufficient information to recognize the **problem of the *inst_dept* schema**.
 - ▶ Because *dept_name* cannot be the primary key for *inst_dept*, the amount of a *budget* may have to be repeated.
 - ▶ This indicates the need to decompose *inst_dept*.



A Lossy Decomposition

- ▶ **Not all decompositions are good.**

- ▶ Suppose we decompose

employee(ID, name, street, city, salary)

into

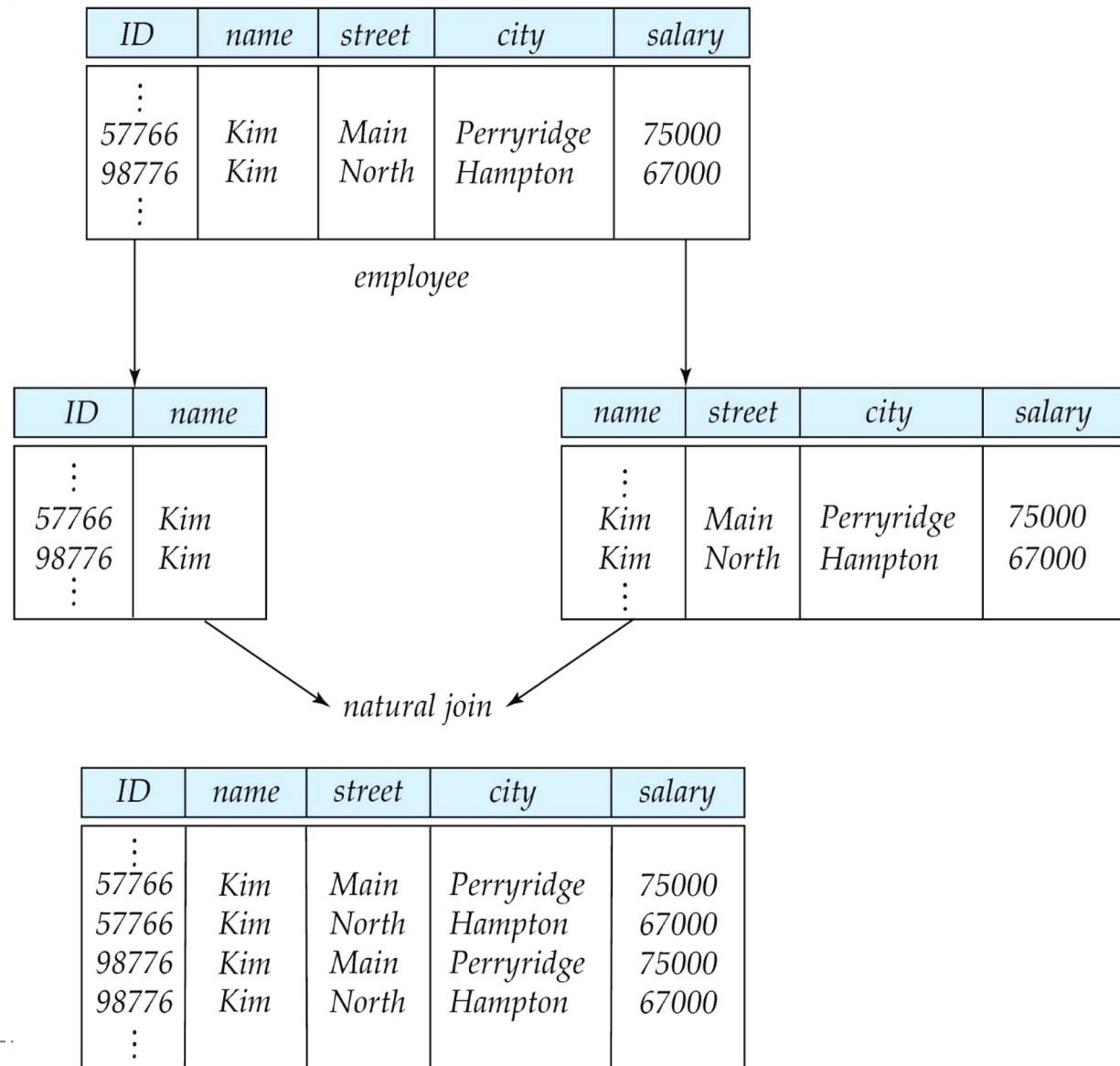
employee1(ID, name)

employee2(name, street, city, salary)



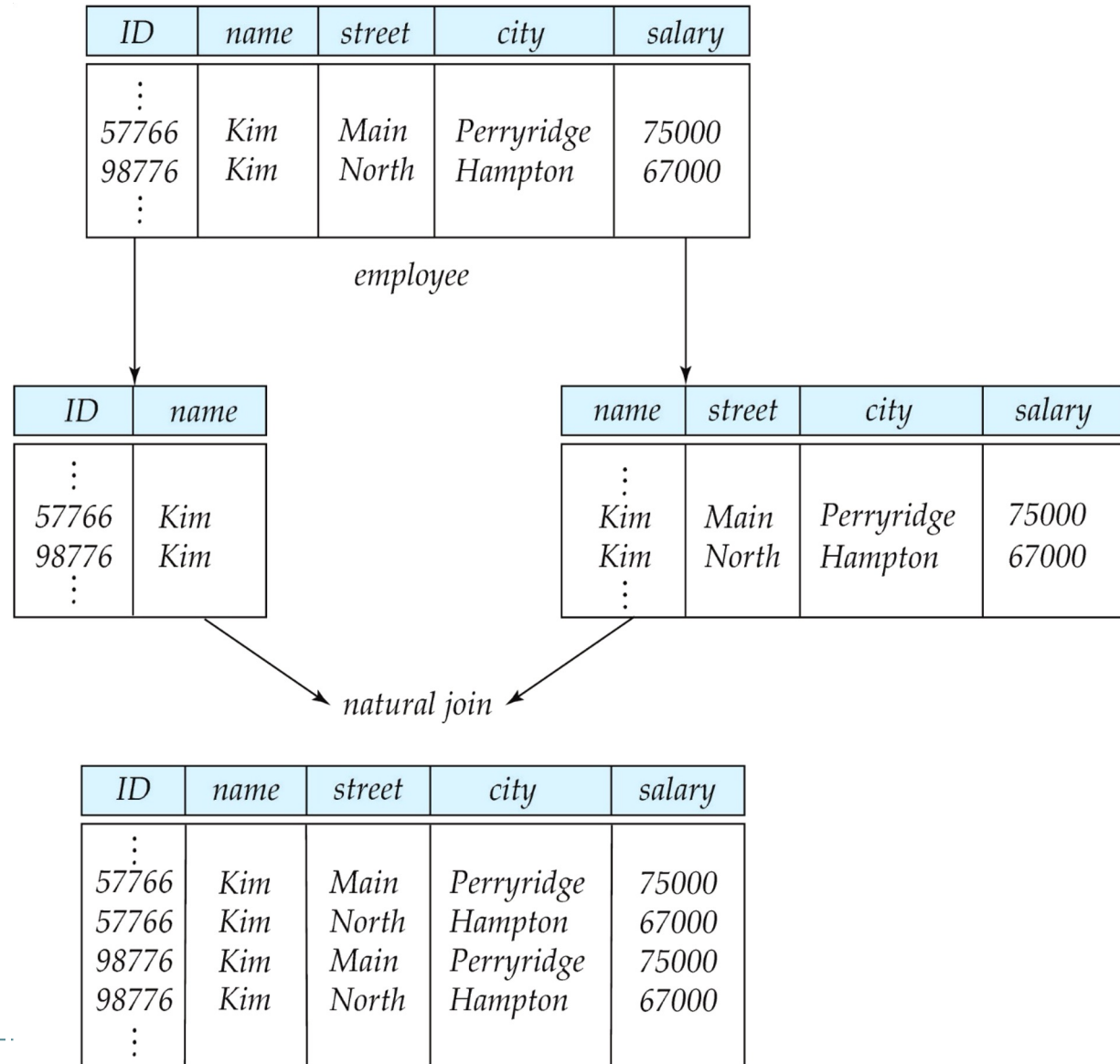
A Lossy Decomposition

- ▶ The two original tuples appear in the result along with two new tuples that *incorrectly mix data values* pertaining to the two employees named *Kim*.
- ▶ We can indicate that a certain *street*, *city*, and *salary* pertain to someone named *Kim*, but we are *unable to distinguish which of the Kims*.



A Lossy Decomposition

- ▶ We cannot *reconstruct* the original *employee* relation -- and so, this is a **lossy decomposition**.



Lossless Decomposition

Definition

- ▶ Let R be a relation schema and let R_1 and R_2 form a decomposition of R . That is $R = R_1 \cup R_2$
- ▶ We say that the decomposition is a **lossless decomposition** if there is no loss of information by replacing R with the two relation schemas $R_1 \cup R_2$
- ▶ Formally,

$$\Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) = r$$

- ▶ And, conversely a decomposition is **lossy** if

$$r \subsetneq \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$



Example of Lossless Decomposition

- Decomposition of $R = (A, B, C)$

$$R_1 = (A, B) \quad R_2 = (B, C)$$

A	B	C
α	1	A
β	2	B

r

A	B
α	1
β	2

$\Pi_{A,B}(r)$

B	C
1	A
2	B

$\Pi_{B,C}(r)$

$\Pi_A(r) \bowtie \Pi_B(r)$

A	B	C
α	1	A
β	2	B



Normalization Theory

- ▶ Decide whether a particular relation R is in **“good” form**.
- ▶ In the case that a relation R is *not in “good” form*, **decompose** it into a set of relations $\{R_1, R_2, \dots, R_n\}$ such that
 - ▶ each relation is in *good form* — There are a number of different forms (called *normal forms*).
 - ▶ the decomposition is a *lossless decomposition*
- ▶ Our theory is based on
 - ▶ **functional dependencies**
 - ▶ **multivalued dependencies**



Decomposition Using Functional Dependencies



Functional Dependencies

- ▶ There are usually a variety of **constraints (rules)** on the data in the real world.
- ▶ For example, some of the constraints that are expected to hold in a university database are:
 - ▶ *Students* and *instructors* are uniquely identified by their *ID*.
 - ▶ Each *instructor* and *student* is (primarily) associated with only one department.
 - ▶ Each *department* has only one value for its *budget*, and only one associated *building*.
- ▶ An instance of a relation that *satisfies all such real-world constraints* is called a **legal instance** of the relation; a legal instance of a database is one where all the relation instances are legal instances.



Functional Dependencies (Cont.)

- ▶ Some of the most commonly used types of real-world **constraints** can be represented formally as *keys* (superkeys, candidate keys, and primary keys), or as *functional dependencies*.
- ▶ A functional dependency is a **generalization** of the notion of a *key*.



Functional Dependencies (Cont.)

Definition

- ▶ Let R be a relation schema, and $\alpha \subseteq R$, $\beta \subseteq R$.
- ▶ The **functional dependency**

$$\alpha \rightarrow \beta$$

holds on R *if and only* if for any legal relations $r(R)$, whenever any two tuples t_1 and t_2 of r agree on the attributes α , they also agree on the attributes β . That is,

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

- ▶ Example: Consider $r(A,B)$ with the following instance of r . On this instance, $A \rightarrow B$ does **NOT** hold, but $B \rightarrow A$ does hold.

A	B
1	4
1	5
3	7



Keys and Functional Dependencies

- ▶ K is a **superkey** for relation schema R if and only if $K \rightarrow R$.
- ▶ K is a **candidate key** for R if and only if $K \rightarrow R$ and **no** $\alpha \subset K, \alpha \rightarrow R$.
- ▶ Functional dependencies allow us to express constraints that **cannot** be expressed using **superkeys**.
 - ▶ Consider the schema:
 $inst_dept (ID, name, salary, dept_name, building, budget)$
 - ▶ $(ID, dept_name)$ forms a **superkey** for $inst_dept$, since
 $ID, dept_name \rightarrow name, salary, building, budget$
 - ▶ Functional dependency $dept_name \rightarrow budget$ holds because for each *department* (identified by $dept_name$) there is a unique budget amount.



Use of Functional Dependencies

- ▶ We use functional dependencies to:
 - ▶ *test relations to see if they are legal under a given set of functional dependencies*
 - ▶ If a relation r is legal under a set F of functional dependencies, we say that r **satisfies (满足)** F .
 - ▶ *specify constraints on the set of legal relations*
 - ▶ We say that F **holds (成立)** on R if all legal relations on R satisfy the set of functional dependencies F .
- ▶ Note: A specific instance of a relation schema **may satisfy** a functional dependency even if the functional dependency does **NOT** hold on **all** legal instances.
 - ▶ For example, a specific instance of *instructor* may, **by chance**, satisfy $name \rightarrow ID$.



Trivial Functional Dependencies

Definition

- ▶ A functional dependency is **trivial (平凡的)** if it is satisfied by *all instances* of a relation.
- ▶ Example:
 - ▶ $ID, name \rightarrow ID$
 - ▶ $name \rightarrow name$
- ▶ In general, $\alpha \rightarrow \beta$ is trivial if $\beta \subseteq \alpha$



Closure of a Set of Functional Dependencies

- ▶ Given a set F of functional dependencies, there are certain other functional dependencies that are *logically implied* (逻辑蕴涵) by F .
 - ▶ For example, given a schema $r(A, B, C)$, If $A \rightarrow B$ and $B \rightarrow C$, hold on r , then we can *infer* that $A \rightarrow C$ must also hold.
- ▶ The set of *all* functional dependencies logically implied by F is the *closure* (闭包) of F .
 - ▶ We denote the *closure* of F by F^+ .
 - ▶ F^+ is a *superset* of F .



Lossless Decomposition and FD

- ▶ We can use functional dependencies to show *when certain decompositions are lossless*.
- ▶ A decomposition of R into R_1 and R_2 is lossless if *at least one of the following dependencies is in F^+* :
 - ▶ $R_1 \cap R_2 \rightarrow R_1$
 - ▶ $R_1 \cap R_2 \rightarrow R_2$
- ▶ In other words, if $R_1 \cap R_2$ forms a **superkey** of either R_1 or R_2 , the decomposition of R is a lossless decomposition.
 - ▶ The above functional dependencies are a *sufficient condition* for **lossless join decomposition**.
 - ▶ The dependencies are a *necessary condition* only if **all constraints are functional dependencies**.



Example

- ▶ Consider the schema:

inst_dept (*ID*, *name*, *salary*, *dept_name*, *building*, *budget*)

- ▶ We decomposed into the *instructor* and *department* schemas:

instructor (*ID*, *name*, *dept_name*, *salary*)

department (*dept_name*, *building*, *budget*)

- ▶ The intersection of these two schemas is *dept_name*.
- ▶ We see that because

dept_name $\rightarrow$ {*dept_name*, *building*, *budget*},

the lossless-decomposition rule is satisfied.



Example

- ▶ $R = \{A, B, C\}$
 $F = \{A \rightarrow B, B \rightarrow C\}$
- ▶ R can be decomposed in different ways
 - ▶ $R_1 = \{A, B\}, R_2 = \{B, C\}$
 - ▶ Lossless decomposition
 - $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$
 - ▶ $R_1 = \{A, B\}, R_2 = \{A, C\}$
 - ▶ Lossless decomposition
 - $R_1 \cap R_2 = \{A\}$ and $A \rightarrow AB$



Normal Forms



Atomic Domains and First Normal Form

- ▶ A domain is **atomic** if its elements are considered to be **indivisible units**.
- ▶ Examples of non-atomic domains:
 - ▶ *A set of names*: such as an attribute *children* in a relation *employee*.
 - ▶ *Composite attributes*: such as an attribute *address* with component attributes *street*, *city*, *state*, and *zip*.
 - ▶ *Identification numbers*: like CS101 that can be divided into smaller units.
- ▶ A **relational schema R** is in **first normal form (1NF)** if *the domains of all attributes of R are atomic*.



First Normal Form

- ▶ The use of **set-valued attributes** can lead to designs with *redundant storage* of data, which in turn can result in *inconsistencies*.
- ▶ Instead of having the relationship *teaches*, set of *instructors* stored with each *section*, and set of *sections* stored with each *instructor*.
- ▶ Whenever data pertaining to which *instructor* teaches which *section* is changed, the *update has to be performed at two places*, otherwise the database would be left in an inconsistent state.
- ▶ Keeping only one of these sets would *avoid repeated information*, but *complicate some queries*.



First Normal Form (Cont.)

- ▶ **Atomicity** is actually a property of *how the elements of the domain are used*.
- ▶ Strings would normally be considered indivisible.
 - ▶ Suppose that students are given roll numbers which are strings of the form *CS0012* or *EE1127*. If the first two characters are extracted to find the department, the domain of roll numbers is not atomic.
 - ▶ *Doing so is a bad idea.*
 - ▶ It leads to encoding of information in *application program* rather than in the database.
 - ▶ Further problems arise if such identification numbers are used as *primary keys*. (*when an employee changes departments, the employee's identification number must be changed everywhere it occurs, which can be a difficult task, or the code that interprets the number would give a wrong result.*)



Boyce-Codd Normal Form

Definition

- ▶ A relation schema R is in **BCNF** with respect to a set F of functional dependencies if for all functional dependencies in F^+ of the form

$$\alpha \rightarrow \beta$$

where $\alpha \subseteq R$ and $\beta \subseteq R$, **at least one** of the following holds:

- ▶ $\alpha \rightarrow \beta$ is trivial (i.e., $\beta \subseteq \alpha$)
- ▶ α is a **superkey** for R
- ▶ Example schema **NOT** in **BCNF**:

inst_dept (ID, name, salary, dept_name, building, budget)

- ▶ $dept_name \rightarrow building, budget$ holds on *inst_dept*, but *dept_name* is **not** a superkey.



Decomposing a Schema into BCNF

- ▶ Let R be a schema that is not in BCNF. Then there is at least one *nontrivial (非平凡的) functional dependency* $\alpha \rightarrow \beta$ such that α is **not** a superkey for R . We replace R in our design with two schemas:

- ▶ $(\alpha \cup \beta)$
- ▶ $(R - (\beta - \alpha))$

- ▶ In the case of $inst_dept$:

$inst_dept(\underline{ID}, name, salary, \underline{dept_name}, building, budget)$

$dept_name \rightarrow building, budget$

- ▶ $\alpha = dept_name$ and $\beta = \{building, budget\}$
- ▶ So $inst_dept$ is replaced by

$(\alpha \cup \beta) = (dept_name, building, budget)$

$(R - (\beta - \alpha)) = (ID, name, salary, dept_name)$



BCNF and Dependency Preservation

- ▶ Constraints, including functional dependencies, are *costly to check in practice* unless they pertain to only one relation.
- ▶ If it is sufficient to test only those dependencies on *each individual relation of a decomposition* in order to ensure that *all functional dependencies hold*, then that decomposition is **dependency preserving**.
- ▶ Because it is not always possible to achieve both **BCNF** and *dependency preservation*, we consider a weaker normal form, known as **third normal form**.



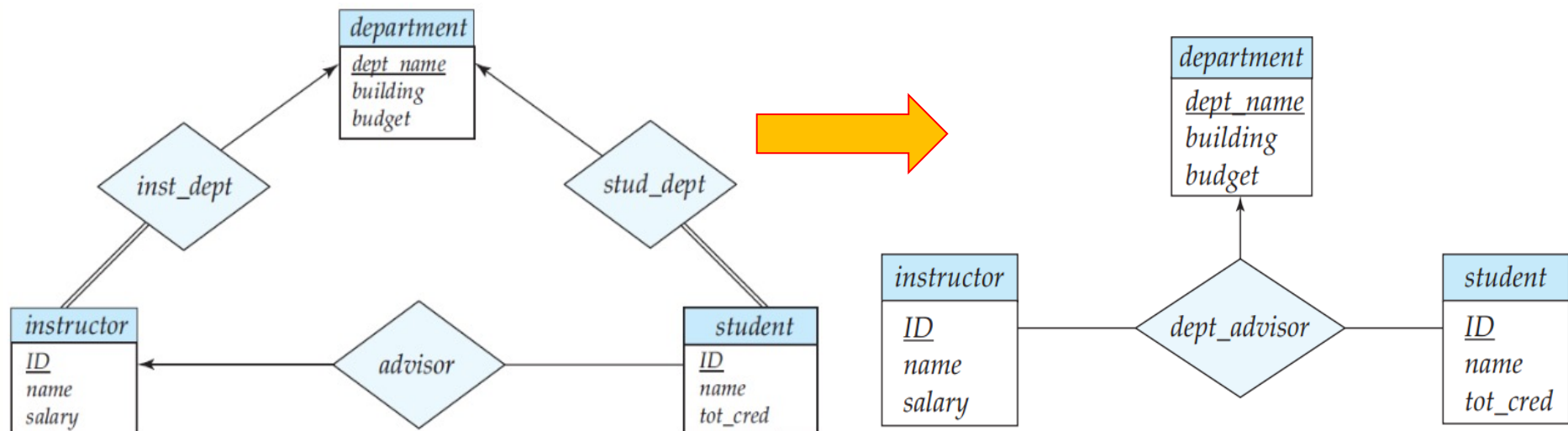
Example

- ▶ $R = \{A, B, C\}$
 $F = \{A \rightarrow B, B \rightarrow C\}$
- ▶ R can be decomposed in different ways
 - ▶ $R_1 = \{A, B\}, R_2 = \{B, C\}$
 - ▶ Lossless decomposition
 - $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$
 - ▶ Dependency preserving
 - ▶ $R_1 = \{A, B\}, R_2 = \{A, C\}$
 - ▶ Lossless decomposition
 - $R_1 \cap R_2 = \{A\}$ and $A \rightarrow AB$
 - ▶ Not dependency preserving
(cannot check $B \rightarrow C$ without computing $R_1 \bowtie R_2$)



Example

- ▶ Suppose that an *instructor* can be associated with only a single *department* and a *student* may have more than one *advisor*, but at most one from a given *department*.
- ▶ One way to implement this change using the *E-R* design is by replacing the *advisor* relationship set with a ternary relationship set, *dept_advisor*, involving entity sets *instructor*, *student*, and *department*.



Example

- ▶ The schema derived from *dept_advisor* is now:

dept_advisor (*s_ID*, *i_ID*, *dept_name*)

- ▶ **Constraint-1:** “*an instructor can act as advisor for only a single department.*” (一位教师只能在单个系中担任导师)

i_ID → *dept_name*

- ▶ **Constraint-2:** “*a student may have more than one advisor, but at most one corresponding to a given department*”. (对于一个给定的系，一名学生可以有至多一位导师)

s_ID, *dept_name* → *i_ID*

- ▶ We see that *dept_advisor* is not in BCNF because *i_ID* is not a **superkey**.



Example

- ▶ Following our rule for BCNF decomposition, we get:

$$\begin{array}{l} i_ID \rightarrow dept_name \\ (s_ID, i_ID) \\ (i_ID, dept_name) \end{array}$$

- ▶ Both the above schemas are BCNF (we can verify that *any schema with only two attributes is in BCNF* by definition).
- ▶ However, there is no schema that includes all the attributes appearing in the functional dependency $s_ID, dept_name \rightarrow i_ID$
- ▶ We say our design is ***not dependency preserving***.
- ▶ Dependency preservation is usually considered desirable.

Third Normal Form

Definition

- ▶ A *relation schema* R is in **third normal form (3NF)** with respect to *a set F of functional dependencies* if, for *all* functional dependencies in F^+ of the form $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, **at least one** of the following holds:
 - ▶ $\alpha \rightarrow \beta$ is a trivial functional dependency
 - ▶ α is a superkey for R
 - ▶ Each attribute A in $\beta - \alpha$ is contained in a candidate key for R
 - ▶ Each attribute A in $\beta - \alpha$ may be contained in a *different* candidate key.
- ▶ Any schema that satisfies BCNF also satisfies 3NF.
- ▶ Third condition is a *minimal relaxation* of BCNF to ensure *dependency preservation* (will see why later).



Third Normal Form

- ▶ Consider again the *dept_advisor* relationship set

dept_advisor (*s_ID*, *i_ID*, *dept_name*)

- ▶ The following functional dependencies hold on it

i_ID $\rightarrow$ *dept_name*

s_ID, *dept_name* $\rightarrow$ *i_ID*

- ▶ The functional dependency *i_ID* $\rightarrow$ *dept_name* caused the *dept_advisor* schema not to be in BCNF.
- ▶ Since the functional dependency *s_ID*, *dept_name* $\rightarrow$ *i_ID* holds on *dept_advisor*, the attribute *dept_name* is contained in a candidate key and, therefore, *dept_advisor* is in 3NF.
 - ▶ Here $\alpha = i_ID$, $\beta = dept_name$, and $\beta - \alpha = dept_name$.



Comparison of BCNF and 3NF

► Comparison

- **Advantages** to 3NF over BCNF. It is always possible to obtain a 3NF design *without sacrificing losslessness or dependency preservation*.
- **Disadvantages** to 3NF.
 - We may have to use **null values** to represent some of the possible meaningful relationships among data items.
 - There is the problem of **repetition of information**.



Comparison of BCNF and 3NF

► Disadvantages to 3NF

► Consider the schema R below, which is in 3NF

- $R = (J, K, L)$
- $F = \{JK \rightarrow L, L \rightarrow K\}$
- And an instance table:

J	L	K
j_1	l_1	k_1
j_2	l_1	k_1
j_3	l_1	k_1
null	l_2	k_2

► What is wrong with the table?

- Repetition of information
- Need to use null values (e.g., to represent the relationship l_2, k_2 where there is no corresponding value for J)



Comparison of BCNF and 3NF

▶ Goal of Normalization

- ▶ In the case that a relation R is *not in “good” form*, **decompose** it into a set of relations $\{R_1, R_2, \dots, R_n\}$ such that
 - ▶ each relation is in *good form* — **BCNF**.
 - ▶ the decomposition is a *lossless decomposition*
 - ▶ **Preferably**, the decomposition should be *dependency preserving*.
- ▶ Since it is *not always possible to satisfy all three*, we may be forced to *choose between BCNF and dependency preservation with 3NF*.



Comparison of BCNF and 3NF

▶ Best Practice in SQL

- ▶ It is worth noting that **SQL** *does not* provide a way of specifying *functional dependencies*, **except** for the special case of declaring superkeys by using the **primary key** or **unique** constraints.
- ▶ Thus even if we had a dependency-preserving decomposition, if we use standard SQL *we can test efficiently only those functional dependencies whose left-hand side is a key*.
- ▶ In summary, even if we are not able to get a dependency-preserving BCNF decomposition, **it is still preferable to opt for BCNF**, *since checking functional dependencies other than primary key constraints is difficult in SQL*.



Higher Normal Forms

- ▶ There are database schemas in BCNF that *do NOT seem to be sufficiently normalized*.

- ▶ Consider a relation

inst_info (ID, child_name, phone)

- ▶ An *instructor* may have more than one *phone* and can have multiple *children*.

ID	child_name	phone
99999	David	512-555-1234
99999	David	512-555-4321
99999	William	512-555-1234
99999	Willian	512-555-4321

inst_info



Higher Normal Forms

- ▶ There are *no non-trivial functional dependencies* and therefore the relation is in BCNF.

<i>ID</i>	<i>child_name</i>	<i>phone</i>
99999	David	512-555-1234
99999	David	512-555-4321
99999	William	512-555-1234
99999	Willian	512-555-4321

inst_info

- ▶ **Insertion anomalies** – i.e., if we add a phone 981-992-3443 to 99999, we need to add two tuples:

(99999, David, 981-992-3443)

(99999, William, 981-992-3443)

- ▶ **Normal forms (like BCNF) based on functional dependencies are not sufficient to deal with situations like this.**



Higher Normal Forms

- ▶ Therefore, it is better to decompose *inst_info* into:

<i>inst_child</i>	<i>ID</i>	<i>child_name</i>
	99999	David
	99999	Willian

<i>inst_phone</i>	<i>ID</i>	<i>phone</i>
	99999	512-555-1234
	99999	512-555-4321

- ▶ This suggests the need for higher normal forms, such as **Fourth Normal Form (4NF)**, which we shall see later (§ 7.6 & 7.7).



Functional-Dependency Theory



Functional-Dependency Theory

- ▶ We now consider the *formal theory* that tells us which functional dependencies are *implied logically* by a given set of functional dependencies.
- ▶ We then develop *algorithms* to *generate lossless decompositions into BCNF and 3NF*.
- ▶ We then develop *algorithms* to *test if a decomposition is dependency-preserving*.



Closure of a Set of Functional Dependencies

- ▶ Given a set F set of functional dependencies, there are certain other functional dependencies that are logically implied by F .
 - ▶ The set of *all functional dependencies logically implied by F* is the **closure of F** (函数依赖集 F 的闭包) .
 - ▶ We denote the closure of F by F^+ .
- ▶ More formally, given a relational schema $r(R)$, a functional dependency f on R is **logically implied (逻辑蕴涵)** by a set of functional dependencies F on r if *every instance of $r(R)$ that satisfies F also satisfies f* .



Example

- ▶ Given a relation schema $r(A, B, C, G, H, I)$ and the set of functional dependencies:

$$A \rightarrow B$$

$$A \rightarrow C$$

$$CG \rightarrow H$$

$$CG \rightarrow I$$

$$B \rightarrow H$$

- ▶ The functional dependency: $A \rightarrow H$ is logically implied.
- ▶ That is whenever a relation satisfies our given set of functional dependencies, $A \rightarrow H$ must also be satisfied by that relation.



Example

- ▶ Suppose that t_1 and t_2 are tuples such that:

$$t_1[A] = t_2[A]$$

- ▶ Given that $A \rightarrow B$, it follows from the definition of functional dependency that:

$$t_1[B] = t_2[B]$$

- ▶ Given that $B \rightarrow H$, it follows from the definition of functional dependency that:

$$t_1[H] = t_2[H]$$

- ▶ Therefore, **whenever t_1 and t_2 are tuples such that $t_1[A] = t_2[A]$, it must be that $t_1[H] = t_2[H]$.**
- ▶ That is exactly the definition of $A \rightarrow H$.



Closure of a Set of Functional Dependencies

- ▶ We can find F^+ , the closure of F , by repeatedly applying **Armstrong's Axioms**:
 - ▶ if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$ (**reflexivity** 自反律)
 - ▶ if $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$ (**augmentation** 增补律)
 - ▶ if $\alpha \rightarrow \beta$, and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$ (**transitivity** 传递律)
- ▶ These rules are
 - ▶ **Sound** (正确有效的) : generate only functional dependencies that *actually hold*;
 - ▶ **Complete** (完备的) : generate *all* functional dependencies that hold.
- ▶ Note: We use Greek letters($\alpha, \beta, \gamma...$) for sets of attributes and $\alpha\beta$ for $\alpha \cup \beta$.



Closure of Functional Dependencies

- ▶ To simplify the computation of F^+ further, we list additional rules:
 - ▶ If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta \gamma$ holds.
(**union**合并律)
 - ▶ If $\alpha \rightarrow \beta \gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds.
(**decomposition**分解律)
 - ▶ If $\alpha \rightarrow \beta$ holds and $\gamma \beta \rightarrow \delta$ holds, then $\alpha \gamma \rightarrow \delta$ holds.
(**pseudotransitivity**伪传递律)
- ▶ The above rules can be inferred from Armstrong's axioms.



Example

- ▶ $R = (A, B, C, G, H, I)$
 $F = \{ A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H \}$
- ▶ Some members of F^+ :
 - ▶ $A \rightarrow H$
 - ▶ by *transitivity* from $A \rightarrow B$ and $B \rightarrow H$
 - ▶ $AG \rightarrow I$
 - ▶ by *augmenting* $A \rightarrow C$ with G , to get $AG \rightarrow CG$ and then transitivity with $CG \rightarrow I$
 - ▶ or by *pseudotransitivity* with $A \rightarrow C$ and $CG \rightarrow I$,
 - ▶ $CG \rightarrow HI$
 - ▶ by *augmenting* $CG \rightarrow I$ to infer $CG \rightarrow CGI$, and *augmenting* of $CG \rightarrow H$ to infer $CGI \rightarrow HI$, and then *transitivity*
 - ▶ or by *union* with $CG \rightarrow H$ and $CG \rightarrow I$



A Procedure to Compute F^+

Opt-1

- ▶ To *compute the closure of a set of functional dependencies F with Armstrong's axioms:*

$F^+ = F$

repeat

for *each functional dependency* f in F^+

 apply *reflexivity* and *augmentation* rules on f

 add the resulting functional dependencies to F^+

for *each pair of functional dependencies* f_1 and f_2 in F^+

if f_1 and f_2 can be combined using *transitivity*

then add the resulting functional dependency to F^+

until F^+ does not change any further

We shall see an alternative procedure for this task later.



A Procedure to Compute F^+

► Analysis

- The left-hand and right-hand sides of a functional dependency are both subsets of R .
- Since a set of size n has 2^n subsets, there are a total of $2^n \times 2^n = 2^{2n}$ **possible** functional dependencies, where *n is the number of attributes in R .*



Closure of Attribute Sets

Definition

- ▶ An attribute B is **functionally determined** (函数决定) by α if $\alpha \rightarrow B$.
- ▶ Given a set of attributes α , define the **closure of α under F** (denoted by α^+) as the set of attributes that are functionally determined by α under F . (属性集闭包)
- ▶ **Algorithm to compute α^+** (吸星大法☺)

$result := \alpha;$

while (**changes to $result$**) **do**

for each $\beta \rightarrow \gamma$ **in** F **do**

begin

if $\beta \subseteq result$ **then** $result := result \cup \gamma,$

end



Example of Attribute Set Closure

▶ $R = (A, B, C, G, H, I)$

▶ $F = \{A \rightarrow B$

$A \rightarrow C$

$CG \rightarrow H$

$CG \rightarrow I$

$B \rightarrow H \}$

▶ **Compute $(AG)^+$:**

1. $result = AG$

2. $result = ABCG$ ($A \rightarrow C$ and $A \rightarrow B$)

3. $result = ABCGH$ ($CG \rightarrow H$ and $CG \subseteq ABCG$)

4. $result = ABCGHI$ ($CG \rightarrow I$ and $CG \subseteq ABCGH$)

▶ **Is AG a candidate key?**

1. Is AG a super key?

1. Does $AG \rightarrow R?$ == Is $(AG)^+ \supseteq R$

2. Is any subset of AG a superkey?

1. Does $A \rightarrow R?$ == Is $(A)^+ \supseteq R$

2. Does $G \rightarrow R?$ == Is $(G)^+ \supseteq R$

3. In general: check for each subset of size $n-1$



Uses of Attribute Set Closure

▶ ① *Testing for superkey*

- ▶ To test if α is a superkey, we compute α^+ , and check if α^+ contains all attributes of R .

▶ ② *Testing functional dependencies*

- ▶ To check if a functional dependency $\alpha \rightarrow \beta$ holds (or, in other words, is in F^+), just check if $\beta \subseteq \alpha^+$.
- ▶ That is, we compute α^+ by using attribute closure, and then check if it contains β .
- ▶ It is a simple and cheap test, and very useful

▶ ③ *Computing closure of F* (一般不用)

- ▶ For each $\gamma \subseteq R$, we find the closure γ^+ , and for each $S \subseteq \gamma^+$, we output a functional dependency $\gamma \rightarrow S$.



Example of Computing F^+

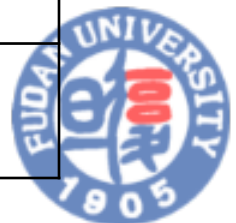
Opt-2

- ▶ Given $F = \{ A \rightarrow B, B \rightarrow C \}$.

Compute F^+ with attributes A, B, C .

- ▶ For each $\gamma \subseteq R$, we find the closure γ^+ .
- ▶ For each $S \subseteq \gamma^+$, we output a functional dependency $\gamma \rightarrow S$.

$A^+ = ABC$	$A \rightarrow A, A \rightarrow B, A \rightarrow C, A \rightarrow AB, A \rightarrow AC, A \rightarrow BC, A \rightarrow ABC$
$B^+ = BC$	$B \rightarrow B, B \rightarrow C, B \rightarrow BC$
$C^+ = C$	$C \rightarrow C$
$AB^+ = ABC$	$AB \rightarrow A, AB \rightarrow B, AB \rightarrow C, AB \rightarrow AB, AB \rightarrow AC, AB \rightarrow BC, AB \rightarrow ABC$
$AC^+ = ABC$	$AC \rightarrow A, AC \rightarrow B, AC \rightarrow C, AC \rightarrow AB, AC \rightarrow AC, AC \rightarrow BC, AC \rightarrow ABC$
$BC^+ = BC$	$BC \rightarrow B, BC \rightarrow C, BC \rightarrow BC$
$ABC^+ = ABC$	$ABC \rightarrow A, ABC \rightarrow B, ABC \rightarrow C, ABC \rightarrow AB, ABC \rightarrow AC, ABC \rightarrow BC, ABC \rightarrow ABC$



Canonical Cover

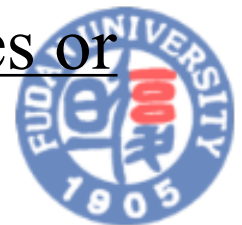
Motivation

- ▶ Suppose that we have a set of functional dependencies F on a relation schema. Whenever a user performs an *update* on the relation, the database system must *ensure that the update does not violate any functional dependencies*; that is, all the functional dependencies in F are satisfied in the new database state.
- ▶ If an update *violates* any functional dependencies in the set F , the system must *roll back* the update.
- ▶ We can *reduce the effort* spent in checking for violations *by testing a simplified set of functional dependencies* that has the same closure as the given set.
- ▶ This simplified set is termed the **canonical cover** (正则覆盖)



Canonical Cover

- ▶ Sets of functional dependencies may have **redundant dependencies** that can be inferred from the others.
 - ▶ E.g.: $A \rightarrow C$ is redundant in: $\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$.
 - ▶ Parts of a functional dependency may be redundant.
 - ▶ E.g.: $\{A \rightarrow B, B \rightarrow C, A \rightarrow \textcolor{red}{C}D\}$ can be simplified to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$
 - ▶ E.g.: $\{A \rightarrow B, B \rightarrow C, A\textcolor{red}{C} \rightarrow D\}$ can be simplified to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$
- ▶ Intuitively, a **canonical cover** (正则覆盖) of F is a “**minimal**” set of functional dependencies (最小依赖集) equivalent to F , having no redundant dependencies or redundant parts of dependencies.



Extraneous Attributes

- ▶ Removing an attribute from the *left side* of a functional dependency could make it a *stronger* constraint. (由弱变强)
 - ▶ For example, if we have $AB \rightarrow C$ and remove B, we get the possibly *stronger* result $A \rightarrow C$. It may be stronger because $A \rightarrow C$ logically implies $AB \rightarrow C$, but $AB \rightarrow C$ does not, on its own, logically imply $A \rightarrow C$
- ▶ **But**, depending on what our set F of functional dependencies happens to be, we may be able to *remove B from $AB \rightarrow C$ safely*.
 - ▶ For example, suppose that
 - ▶ $F = \{AB \rightarrow C, A \rightarrow D, D \rightarrow C\}$
 - ▶ Then we can show that F logically implies $A \rightarrow C$, making extraneous in $AB \rightarrow C$.



Extraneous Attributes

- ▶ Removing an attribute from the *right side* of a functional dependency could make it a *weaker* constraint. (由强变弱)
 - ▶ For example, if we have $AB \rightarrow CD$ and remove C , we get the possibly *weaker* result $AB \rightarrow D$. It may be weaker because using just $AB \rightarrow D$, we can no longer infer $AB \rightarrow C$.
- ▶ **But**, depending on what our set F of functional dependencies happens to be, we may be able to *remove C from $AB \rightarrow CD$ safely*.
 - ▶ For example, suppose that
$$F = \{ AB \rightarrow CD, A \rightarrow C \}$$
 - ▶ Then we can show that even after replacing $AB \rightarrow CD$ by $AB \rightarrow D$, we can still infer $AB \rightarrow C$ and thus $AB \rightarrow CD$.



Extraneous Attributes

Definition

- ▶ Consider a set F of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in F .
- ▶ Attribute A is **extraneous** (无关的) in α if $A \in \alpha$ and F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
- ▶ Attribute A is **extraneous** (无关的) in β if $A \in \beta$ and the set of functional dependencies $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies F .
- ▶ **Note**: Implication in the opposite direction is trivial in each of the cases above, since a **stronger** functional dependency always implies a **weaker** one.



Testing if an Attribute is Extraneous

Testing

- ▶ Consider a set F of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in F .
- ▶ *To test if attribute $A \in \alpha$ is extraneous in α*
 1. compute $(\{\alpha\} - A)^+$ using the dependencies in F
 2. check that $(\{\alpha\} - A)^+$ contains β ; if it does, A is extraneous in α .
- ▶ *To test if attribute $A \in \beta$ is extraneous in β*
 1. compute α^+ using only the dependencies in
$$F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\},$$
 2. check that α^+ contains A ; if it does, A is extraneous in β



Extraneous Attributes - Example

- ▶ Let $F = \{AB \rightarrow CD, A \rightarrow E, E \rightarrow C\}$
- ▶ To check if C is extraneous in $AB \rightarrow CD$, we:
 - ▶ Compute the attribute closure of AB ($\{AB\}^+$) under F'
 $= \{AB \rightarrow D, A \rightarrow E, E \rightarrow C\}$
 - ▶ The closure is $\{AB\}^+ = ABCDE$, which includes CD
 - ▶ This implies that C is extraneous



Canonical Cover

Definition

- ▶ A **canonical cover** F_c for F is a set of dependencies such that
 - ▶ F logically implies all dependencies in F_c .
 - ▶ F_c logically implies all dependencies in F . } $F_c^+ = F^+$
- ▶ *No functional dependency in F_c contains an extraneous attribute.*
- ▶ Each **left side** of a functional dependency in F_c is **unique** (that is, there are no two dependencies $\alpha_1 \rightarrow \beta_1$ and $\alpha_2 \rightarrow \beta_2$ in F_c such that $\alpha_1 = \alpha_2$)



Computing a Canonical Cover

$F_c = F$

repeat

Use the *union rule* to replace any dependencies in F

$\alpha_1 \rightarrow \beta_1$ and $\alpha_1 \rightarrow \beta_2$ with $\alpha_1 \rightarrow \beta_1 \beta_2$

Find a functional dependency $\alpha \rightarrow \beta$ in F_c with an

extraneous attribute either in α or in β

/ Note: test for extraneous attributes done using F_c , not F */*

If an extraneous attribute is found, delete it from $\alpha \rightarrow \beta$

until F_c does not change

- **Note:** Union rule may become applicable after some extraneous attributes have been deleted, so it has to be *re-applied*.



Computing a Canonical Cover - Example

- ▶ $R = (A, B, C)$
 $F = \{ A \rightarrow BC$
 $B \rightarrow C$
 $A \rightarrow B$
 $AB \rightarrow C \}$
- ▶ Combine $A \rightarrow BC$ and $A \rightarrow B$ into $A \rightarrow BC$
 - ▶ Set is now $\{A \rightarrow BC, B \rightarrow C, AB \rightarrow C\}$
- ▶ **A is extraneous in $AB \rightarrow C$**
 - ▶ Check if the result of deleting A from $AB \rightarrow C$ is implied by the other dependencies
 - ▶ Yes: in fact, $B \rightarrow C$ is already present!
 - ▶ Set is now $\{A \rightarrow BC, B \rightarrow C\}$
- ▶ **C is extraneous in $A \rightarrow BC$**
 - ▶ Check if $A \rightarrow C$ is logically implied by $A \rightarrow B$ and the other dependencies
 - ▶ Yes: using transitivity on $A \rightarrow B$ and $B \rightarrow C$.
- ▶ **The canonical cover is:**
 $A \rightarrow B$
 $B \rightarrow C$



Computing a Canonical Cover - Example

- ▶ **A canonical cover might not be unique.**
- ▶ Consider $F = \{A \rightarrow BC, B \rightarrow AC, C \rightarrow AB\}$.
- ▶ If we apply the extraneity test to $A \rightarrow BC$, we find that both B and C are extraneous under F .
- ▶ It is *incorrect to delete both*! The algorithm for finding the canonical cover *picks one of the two*, and deletes it.
- ▶ **If C is deleted**, we get $F' = \{A \rightarrow B, B \rightarrow AC, C \rightarrow AB\}$.
 - ▶ Now, B is not extraneous in the side of $A \rightarrow B$ under F' .
 - ▶ Continue: A and B are extraneous in the right-hand side of $C \rightarrow AB$, which leads to two canonical covers:
 - ▶ $F_c = \{A \rightarrow B, B \rightarrow C, C \rightarrow A\}$
 - ▶ $F_c = \{A \rightarrow B, B \rightarrow AC, C \rightarrow B\}$.
- ▶ **If B is deleted**, we get $F' = \{A \rightarrow C, B \rightarrow AC, C \rightarrow AB\}$.
 - ▶ This case is symmetrical to the previous case, leading to
 - ▶ $F_c = \{A \rightarrow C, C \rightarrow B, B \rightarrow A\}$
 - ▶ $F_c = \{A \rightarrow C, B \rightarrow C, C \rightarrow AB\}$.



Dependency Preservation

- ▶ Let F be a set of functional dependencies on a schema R , and let $R_1, R_2, \dots, R_n$ be a decomposition of R .
- ▶ The *restriction of F to R_i* is the set F_i of all functional dependencies in F^+ that include *only attributes in R_i* .
- ▶ When all functional dependencies in a **restriction** involve attributes of only one relation schema, it is *possible to test such a dependency for satisfaction by checking only one relation*.
- ▶ The definition of restriction uses *all dependencies in F^+* , not just those in F (Example: Suppose $F = \{A \rightarrow B, B \rightarrow C\}$, and we have a decomposition into AC and AB . The restriction of F to AC includes $A \rightarrow C$.)



Dependency Preservation

Definition

- ▶ The set of restrictions $F_1, F_2, \dots, F_n$ is the set of dependencies that can be checked efficiently. We now must ask *whether testing only the restrictions is sufficient*.
- ▶ Let $F' = F_1 \cup F_2 \cup \dots \cup F_n$. F' is a set of functional dependencies on schema R . In general, $F' \neq F$. But it may be that $F'^+ = F^+$.
- ▶ A decomposition is **dependency preserving**, if it has the property $F'^+ = F^+$.
 - ▶ If $F'^+ = F^+$, every dependency in F is logically implied by F' , and, if we verify that F' is satisfied, we have verified that F is satisfied.
 - ▶ If $F'^+ \neq F^+$, then checking updates for violation of functional dependencies *may require computing joins*, which is expensive.

Test for Dependency Preservation

Input:

A decomposition
 $D = \{R_1, R_2, \dots, R_n\}$
and a set F of
functional
dependencies.

```
compute  $F^+$ ;  
for each schema  $R_i$  in  $D$  do  
  begin  
     $F_i :=$  the restriction of  $F^+$  to  $R_i$ ;  
  end  
 $F' := \emptyset$   
for each restriction  $F_i$  do  
  begin  
     $F' = F' \cup F_i$   
  end  
compute  $F'^+$ ;  
if ( $F'^+ = F^+$ ) then return (true)  
  else return (false);
```

Compute F^+

Compute each F_i

Compute $F' = F_1 \cup F_2 \cup \dots \cup F_n$

- ▶ This algorithm takes **exponential time** since it requires computation of F^+ . So we shall consider an **alternative approach**.



Test for Dependency Preservation

- ▶ **Alternative 1:** If *each member of F can be tested on one of the relations of the decomposition*, then the decomposition is dependency preserving (an easy way to show dependency preservation); however, *it does not always work*.
- ▶ There are cases where, even though the decomposition is dependency preserving, there is **a dependency in F** that *cannot be tested in any one relation in the decomposition*.
- ▶ This alternative can be used only as a *sufficient condition* that is easy to check; if it fails we cannot conclude that the decomposition is not dependency preserving; instead we will have to apply the *general test*.



Test for Dependency Preservation

- **Alternative 2:** To check if a dependency $\alpha \rightarrow \beta$ in F is preserved in a decomposition of R into $R_1, R_2, \dots, R_n$:

(the attribute closure is done with respect to F)

result = α

repeat

for each R_i in the decomposition

$t = (result \cap R_i)^+ \cap R_i$
 $result = result \cup t$

until (no change to *result*)

It equals to **compute**
 $result^+$ under F_i .

Repeating it gives
 $result^+$ under F .

$$result = \alpha_{\cup_i F_i}^+$$

- If *result* contains **all attributes in β** ($\beta \subseteq \alpha_{\cup_i F_i}^+$), then the functional dependency $\alpha \rightarrow \beta$ is preserved ($\alpha \rightarrow \beta \in (\cup_i F_i)^+ = F'^+$).
- The decomposition is dependency preserving **if and only if** the procedure shows that all the dependencies in F are preserved.

Test for Dependency Preservation

- ▶ The key idea behind the test: **testing each functional dependency $\alpha \rightarrow \beta$ in F to see if it is preserved in F'**
 - ▶ To do so, we compute the *closure of α under F'* ; the dependency is preserved exactly when *the closure includes β* .
 - ▶ The decomposition is dependency preserving *if and only if* all the dependencies in F are found to be preserved.



Test for Dependency Preservation

- ▶ The key idea behind the test: **use a modified form of the attribute-closure algorithm to compute closure under F'** , without actually first computing F'
- ▶ We wish to avoid computing F' since it is quite expensive. Note that $F' = F_1 \cup F_2 \cup \dots \cup F_n$, where F_i is the restriction of F on R_i .

$$t = (result \cap R_i)^+ \cap R_i$$
$$result = result \cup t$$
- ▶ The algorithm computes the **attribute closure of $(result \cap R_i)$** with respect to F , **intersects the closure with R_i** , and adds the resultant set of attributes to $result$; this sequence of steps **is equivalent to computing the closure of $result$ under F_i** . Repeating this step for each i inside the loop gives the closure of $result$ under F' .



Test for Dependency Preservation

- ▶ **Correctness** of the **attribute-closure approach**
 - ▶ For any $\gamma \subseteq R_i$, $\gamma \rightarrow \gamma^+$ is a functional dependency in F^+ .
 - ▶ $\gamma \rightarrow (\gamma^+ \cap R_i)$ is a functional dependency that is in F_i , the restriction of F^+ to R_i .
 - ▶ Conversely, if $\gamma \rightarrow \delta$ were in F_i , then δ would be a subset of $\gamma^+ \cap R_i$.
- ▶ This procedure takes **polynomial time**, instead of the **exponential time** required to compute F^+ and F'^{+} .



Example

- ▶ $R = \{A, B, C\}$
 $F = \{A \rightarrow B, \textcolor{violet}{B} \rightarrow \textcolor{violet}{C}\}$
- ▶ R can be decomposed in different ways
 - ▶ $R_1 = \{A, B\}, R_2 = \{B, C\}$
 - ▶ Lossless-join decomposition
 - $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$
 - ▶ *Dependency preserving*
 - ▶ $R_1 = \{A, B\}, R_2 = \{\textcolor{violet}{A}, \textcolor{violet}{C}\}$
 - ▶ Lossless-join decomposition
 - $R_1 \cap R_2 = \{A\}$ and $A \rightarrow AB$
 - ▶ *Not dependency preserving*
 - Cannot check $B \rightarrow C$ without computing $R_1 \bowtie R_2$



Algorithms for Decomposition Using Functional Dependencies



BCNF Decomposition

Review

- ▶ A relation schema R is in **BCNF** with respect to a set F of functional dependencies if for **all** functional dependencies in F^+ of the form $\alpha \rightarrow \beta$ where $\alpha \subseteq R$ and $\beta \subseteq R$, *at least one* of the following holds:
 - ▶ $\alpha \rightarrow \beta$ is trivial;
 - ▶ α is a superkey for R .
- ▶ The definition of BCNF can be used directly to test if a relation is in BCNF.
- ▶ However, *computation of F^+* can be a tedious task.



Testing for BCNF

- ▶ *To check if a **nontrivial** dependency $\alpha \rightarrow \beta$ causes a violation of BCNF*
 - ▶ Compute α^+ ;
 - ▶ Verify that it includes all attributes of R ; that is, it is a superkey of R .
- ▶ *To check if a relation schema R is in BCNF*
 - ▶ **Simplified test:** *check only the dependencies in the given set F for violation of BCNF, rather than check all dependencies in F^+ .*
 - ▶ If none of the dependencies in F causes a violation of BCNF, then none of the dependencies in F^+ will cause a violation of BCNF either.



Testing for BCNF

- ▶ However, **simplified test using only F does NOT work when testing a relation R_i , in a decomposition of R .**
- ▶ Consider $R (A, B, C, D, E)$, with $F = \{A \rightarrow B, BC \rightarrow D\}$
 - ▶ Decompose R into $R_1 (A, B)$ and $R_2 (A, C, D, E)$.
 - ▶ *Neither of the dependencies in F contains only attributes from (A, C, D, E) .*
 - ▶ *We might be misled into thinking R_2 satisfies BCNF.*
 - ▶ In fact $AC \rightarrow D$ (by pseudotransitivity) in F^+ that shows that R_2 is not in BCNF.
- ▶ *We may need a dependency that is in F^+ , but is NOT in F , to show that a decomposed relation is not in BCNF.*



Testing Decomposition for BCNF

- ▶ *To check if a relation R_i in a decomposition of R is in BCNF*

- ▶ Either test R_i for BCNF with respect to the *restriction* of F to R_i .
The **restriction of F to R_i** is the set F_i of all functional dependencies in F^+ that include **only attributes in R_i**



Compute F^+

- ▶ Or use the *original set* of dependencies F that hold on R , but with the following test:
 - ▶ For every set of attributes $\alpha \subseteq R_i$, check that α^+ (the attribute closure of α under F) *either includes no attribute of R_i - α (trivial), or includes all attributes of R_i (superkey).*
 - ▶ If the condition is violated by some set of attributes α in R_i , the dependency

$$\alpha \rightarrow (\alpha^+ - \alpha) \cap R_i$$

can be shown to be present in F^+ , showing R_i violates BCNF.

BCNF Decomposition Algorithm

$result := \{R\};$

$done := false;$

while (not $done$) **do**

if (there is a schema R_i in $result$ that is not in BCNF)

then begin

 let $\alpha \rightarrow \beta$ be a **nontrivial functional dependency** that
 holds on R_i such that $\alpha \rightarrow R_i$ is not in F^+ ,
 and $\alpha \cap \beta = \emptyset$;

▶ $(\alpha \cup \beta)$

▶ $(R - (\beta - \alpha))$

$result := (result - R_i) \cup (R_i - \beta) \cup (\alpha, \beta);$

end

else $done := true;$

Using dependencies that demonstrate violation of BCNF to perform the decomposition

The decomposition that the algorithm generates is not only in BCNF, but is also a lossless decomposition.

Note: Each R_i is in **BCNF**, and the decomposition is **lossless**. When we replace a schema R_i with $(R_i - \beta)$ and (α, β) , the dependency $\alpha \rightarrow \beta$ holds, and $(R_i - \beta) \cap (\alpha, \beta) = \alpha$.

▶ A decomposition of R into R_1 and R_2 is lossless if **at least one of the following dependencies is in F^+**

▶ $R_1 \cap R_2 \rightarrow R_1$

▶ $R_1 \cap R_2 \rightarrow R_2$

BCNF Decomposition - Example

- ▶ In the case of *inst_dept* :

inst_dept (*ID*, *name*, *salary*, *dept_name*, *building*, *budget*)

dept_name $\rightarrow$ *building*, *budget*

- ▶ $\alpha = \text{dept_name}$ and $\beta = \{\text{building}, \text{budget}\}$
 - ▶ The functional dependency satisfies $\alpha \cap \beta = \emptyset$.
- ▶ So *inst_dept* is replaced by
 - $(\alpha, \beta) = (\text{dept_name}, \text{building}, \text{budget})$
 - $(R - \beta) = (ID, \text{name}, \text{salary}, \text{dept_name})$
- ▶ If we did not require $\alpha \cap \beta = \emptyset$, then those attributes in $\alpha \cap \beta$ would not appear in the schema $(R_i - \beta)$.
 - ▶ $(R_i - \beta) \cap (\alpha, \beta) \rightarrow \beta$ (i.e. $\alpha \rightarrow \beta$) would no longer hold.



BCNF Decomposition - Example

class (*course_id*, *title*, *dept_name*, *credits*, *sec_id*, *semester*, *year*,
building, *room_number*, *capacity*, *time_slot_id*)

course_id $\rightarrow$ *title*, *dept_name*, *credits*

building, *room_number* $\rightarrow$ *capacity*

course_id, *sec_id*, *semester*, *year* $\rightarrow$ *building*, *room_number*, *time_slot_id*

course_id $\rightarrow$ *title*, *dept_name*, *credits* holds
but *course_id* is not a superkey.

course (*course_id*, *title*, *dept_name*, *credits*)

class-1 (*course_id*, *sec_id*, *semester*, *year*,
building, *room_number*, *capacity*, *time_slot_id*)

building, *room_number* $\rightarrow$ *capacity* holds on *class-1*,
but {*building*, *room_number*} is not a superkey for *class-1*

classroom (*building*, *room_number*, *capacity*)

section (*course_id*, *sec_id*, *semester*, *year*,
building, *room_number*, *time_slot_id*)

BCNF and Dependency Preservation

- ▶ *It is not always possible to get a BCNF decomposition that is dependency preserving.*
- ▶ Example: $R = (J, K, L)$, $F = \{JK \rightarrow L, L \rightarrow K\}$
 - ▶ Two candidate keys are JK and JL
 - ▶ R is not in BCNF.
 - ▶ Any decomposition of R will fail to preserve
$$JK \rightarrow L$$
 - ▶ This implies that testing for $JK \rightarrow L$ *requires a join*.



Third Normal Form: Motivation

- ▶ There are some situations where
 - ▶ BCNF is *not dependency preserving*, and
 - ▶ *Efficient checking for function dependency violation* on updates is important.
- ▶ Solution: define a **weaker** normal form-- **Third Normal Form (3NF)**
 - ▶ Allows *some redundancy*
 - ▶ But functional dependencies can be *checked on individual relations* without computing a join
- ▶ There is *always a lossless-join, dependency-preserving decomposition into 3NF*.



Third Normal Form

Review

- ▶ A relation schema R is in **3NF** with respect to a set F of functional dependencies if, for all functional dependencies in F^+ of the form $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, *at least one* of the following holds:
 - ▶ $\alpha \rightarrow \beta$ is a trivial functional dependency.
 - ▶ α is a superkey for R .
 - ▶ Each attribute A in $\beta - \alpha$ is contained in a candidate key for R .
 - ▶ Each attribute A in $\beta - \alpha$ may be contained in a *different candidate key*.
 - ▶ It represents, in some sense, a *minimal relaxation* of the BCNF conditions that helps ensure that every schema has a *dependency preserving decomposition* into 3NF.

Testing for 3NF

- ▶ **Optimization goal:** Need to check *only FDs in F* , need not check all FDs in F^+ .
- ▶ Use attribute closure to check for each dependency $\alpha \rightarrow \beta$, *if α is a superkey*.
- ▶ *If α is not a superkey*, we have to verify *if each attribute in β is contained in a candidate key of R* .
 - ▶ This test is rather more expensive, since it involve finding candidate keys.
 - ▶ Testing for 3NF has been shown to be **NP-hard**.
 - ▶ Interestingly, *decomposition into third normal form* (described shortly) can be done in polynomial time.



3NF Decomposition Algorithm

Let F_c be a canonical cover for F ;

Compute F_c

$i := 0$;

for each functional dependency $\alpha \rightarrow \beta$ in F_c

$i := i + 1$;

$R_i := \alpha\beta$;

Explicitly building a schema
for each dependency in F_c

if none of the schemas R_j , $1 \leq j \leq i$ contains **a candidate key** for R

then begin

$i := i + 1$;

$R_i := \text{any candidate key for } R$;

Guarantee that at least one schema
contains a candidate key for R .

end

repeat

if any schema R_j is **contained in another schema** R_k

then

/ delete R_j */*

$R_j := R_i$;

$i := i - 1$;

Remove redundant
relations

until no more R_j s can be deleted

return $(R_1, R_2, \dots, R_i)$



3NF Decomposition Algorithm (Cont.)

- ▶ This algorithm is also called the **3NF synthesis algorithm**.

合成算法
- ▶ It takes *a set of dependencies* and **adds one schema at a time, instead of decomposing** the initial schema repeatedly.
- ▶ This algorithm ensures
 - ▶ Each relation schema R_i is in **3NF**.
 - ▶ Decomposition is *dependency preserving* and *lossless-join*.
 - ▶ Proof of correctness is introduced in the textbook, but beyond our presentation.



3NF Decomposition - Example

cust_banker_branch (customer_id, employee_id, branch_name, type)

customer_id, employee_id $\rightarrow$ branch_name, type

employee_id $\rightarrow$ branch_name

customer_id, branch_name $\rightarrow$ employee_id

Compute F_c : *branch_name* is extraneous in the first dependency, and no other attribute is extraneous

customer_id, employee_id $\rightarrow$ type

employee_id $\rightarrow$ branch_name

customer_id, branch_name $\rightarrow$ employee_id

**for loop generates
3NF schema**

(customer_id, employee_id, type)

(employee_id, branch_name)

(customer_id, branch_name, employee_id)

(customer_id, employee_id, type) contains a candidate key of the original schema, so no further relation schema needs be added.

(employee_id, branch_name) is redundant.

(customer_id, employee_id, type)

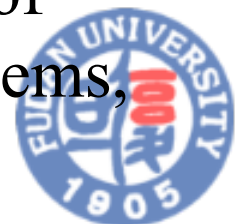
(customer_id, branch_name, employee_id)



Comparison of BCNF and 3NF

Review

- ▶ It is **always possible** to decompose a relation into a set of relations that are in **3NF** such that
 - ▶ the decomposition is *lossless*
 - ▶ the dependencies are *preserved*
- ▶ It is **always possible** to decompose a relation into a set of relations that are in **BCNF** such that
 - ▶ the decomposition is *lossless*.
 - ▶ it may **NOT** be possible to *preserve dependencies*.
- ▶ Nevertheless, there are **disadvantages** to 3NF
 - ▶ We may have to use *null values* to represent some of the possible meaningful relationships among data items,
 - ▶ There is the problem of *repetition of information*.



Design Goals

Review

- ▶ **Goals** for a relational database design are
 - ▶ *BCNF; lossless decomposition; dependency preservation*
- ▶ If we cannot achieve this, we accept **one of**
 - ▶ *Lack of dependency preservation*
 - ▶ *Redundancy due to use of 3NF*
- ▶ Interestingly, SQL does not provide a direct way of specifying functional dependencies other than **superkeys**.
 - ▶ Can specify FDs using assertions, but they are expensive to test, and currently not supported by any of the widely used databases!
- ▶ Even if we had a *dependency preserving decomposition*, using SQL we would not be able to efficiently test a functional dependency whose *left hand side is not a key*.



Decomposition Using Multivalued Dependencies



Multivalued Dependencies

- ▶ Suppose we record names of children, and phone numbers for instructors
 - ▶ *inst_child (ID, child_name)*
 - ▶ *inst_phone (ID, phone_number)*
- ▶ If we were to combine these schemas to get
 - ▶ *inst_info (ID, child_name, phone_number)*
 - ▶ Example data:
 - (99999, David, 512-555-1234)
 - (99999, David, 512-555-4321)
 - (99999, William, 512-555-1234)
 - (99999, William, 512-555-4321)
- ▶ This relation is in BCNF (there are *no non-trivial functional dependencies*) but there is redundancy.



Multivalued Dependencies (MVDs) Definition

- Let R be a relation schema and let $\alpha \subseteq R$ and $\beta \subseteq R$. The **multivalued dependency**

$$\alpha \twoheadrightarrow \beta$$

holds on R if in any legal relation $r(R)$, for **all pairs** for tuples t_1 and t_2 in r such that $t_1[\alpha] = t_2[\alpha]$, there exist tuples t_3 and t_4 in r such that:

$$t_1[\alpha] = t_2[\alpha] = t_3[\alpha] = t_4[\alpha]$$

$$t_3[\beta] = t_1[\beta]$$

$$t_3[R - \beta] = t_2[R - \beta]$$

$$t_4[\beta] = t_2[\beta]$$

$$t_4[R - \beta] = t_1[R - \beta]$$

	α	β	$R - \alpha - \beta$
t_1	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$a_{j+1} \dots a_n$
t_2	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$b_{j+1} \dots b_n$
t_3	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$b_{j+1} \dots b_n$
t_4	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$a_{j+1} \dots a_n$

Multivalued Dependencies

- ▶ If $\beta \subseteq \alpha$ or $\beta \cup \alpha = R$, $\alpha \twoheadrightarrow \beta$ is trivial.
- ▶ From the definition of multivalued dependency, we can derive the following rule:

- ▶ **If $\alpha \rightarrow \beta$, then $\alpha \twoheadrightarrow \beta$**

That is, every functional dependency is also a multivalued dependency.

- ▶ **If $\alpha \twoheadrightarrow \beta$, then $\alpha \twoheadrightarrow R - \alpha - \beta$**



Example

- ▶ Let R be a relation schema with a set of attributes that are partitioned into three nonempty subsets.

$$Y, Z, W$$

- ▶ We say that $Y \twoheadrightarrow Z$ (**Y multidetermines Z**) if and only if for all possible relations $r(R)$

$$\langle y_1, z_1, w_1 \rangle \in r \text{ and } \langle y_1, z_2, w_2 \rangle \in r$$

then

$$\langle y_1, z_1, w_2 \rangle \in r \text{ and } \langle y_1, z_2, w_1 \rangle \in r$$

- ▶ Note that since the behavior of Z and W are identical it follows that

$$Y \twoheadrightarrow Z \text{ if } Y \twoheadrightarrow W$$



Example (Cont.)

- ▶ In the case of *inst_info* (*ID*, *child_name*, *phone_number*):

$$ID \twoheadrightarrow child_name$$

$$ID \twoheadrightarrow phone_number$$

- ▶ The above formal definition is supposed to formalize the notion that given *a particular value* of *Y* (*ID*), it has associated with it *a set of values* of *Z* (*child_name*) and *a set of values* of *W* (*phone_number*), and *these two sets are in some sense independent of each other*.
- ▶ Note:
 - ▶ If $Y \rightarrow Z$ then $Y \twoheadrightarrow Z$
 - ▶ Indeed we have (in above notation) $z_1 = z_2$
The claim follows.



Use of Multivalued Dependencies

- ▶ We **use multivalued dependencies** in two ways:
 1. *To test relations to determine whether they are legal under a given set of functional and multivalued dependencies*
 2. *To specify constraints on the set of legal relations.* We shall thus concern ourselves **only** with relations that satisfy a given set of functional and multivalued dependencies.
- ▶ If a relation r fails to satisfy a given multivalued dependency, we can construct a relations r' that does satisfy the multivalued dependency *by adding tuples to r .*



Theory of MVDs

- ▶ The **closure** D^+ of D is the set of *all functional and multivalued dependencies* logically implied by D .
 - ▶ We can compute D^+ from D , using the formal definitions of functional dependencies and multivalued dependencies.
 - ▶ We can manage with such reasoning for very simple multivalued dependencies, which seem to be most common in practice.
 - ▶ For complex dependencies, it is better to reason about sets of dependencies using a system of inference rules.



Fourth Normal Form

Definition

- ▶ A relation schema R is in **4NF** with respect to a set D of *functional and multivalued dependencies* if for **all** multivalued dependencies in D^+ of the form $\alpha \twoheadrightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, **at least one** of the following hold:
 - ▶ $\alpha \twoheadrightarrow \beta$ is **trivial** (i.e., $\beta \subseteq \alpha$ **or** $\alpha \cup \beta = R$)
 - ▶ α is a **superkey for schema R**
- ▶ The definition of 4NF differs from the definition of BCNF in only *the use of multivalued dependencies*.
 - ▶ *If a relation is in 4NF, it is in BCNF.*
 - ▶ If a schema $r(R)$ is not in BCNF, then there is a nontrivial functional dependency $\alpha \rightarrow \beta$ holding on R , where α is not a superkey. Since $\alpha \rightarrow \beta$ implies $\alpha \twoheadrightarrow \beta$, $r(R)$ cannot be in 4NF.



Testing for 4NF

- ▶ Let $r(R)$ be a relation schema, and let $r_1(R_1), r_2(R_2), \dots, r_n(R_n)$ be a decomposition of $r(R)$.
- ▶ *To check if each relation schema r_i in the decomposition is in 4NF*: find what multivalued dependencies hold on each r_i .
- ▶ Consider a set D of both functional and multivalued dependencies. The **restriction** of D to R_i is the set D_i consisting of:
 - ▶ All functional dependencies in D^+ that include only attributes of R_i .
 - ▶ All multivalued dependencies of the form:

$$\alpha \twoheadrightarrow \beta \cap R_i$$

where $\alpha \subseteq R_i$ and $\alpha \twoheadrightarrow \beta$ is in D^+ .



4NF Decomposition Algorithm

result := {*R*};

done := false;

compute D^+ ; let D_i denote the restriction of D^+ to R_i

while (**not** *done*) **do**

if (there is a schema R_i in *result* that is not in 4NF) **then**
 begin

 let $\alpha \twoheadrightarrow \beta$ be a nontrivial multivalued dependency that holds on R_i such that $\alpha \rightarrow R_i$ is not in D_i , and $\alpha \cap \beta = \emptyset$;

result := (*result* - R_i) $\cup$ ($R_i - \beta$) $\cup$ (α, β)

end

else *done* := true;

- ▶ Note: each R_i is in 4NF, and decomposition is *lossless-join*.
- ▶ It is *identical to the BCNF decomposition algorithm*, **except** that it uses multivalued dependencies and uses the restriction of D^+ to R_i .



Example

- ▶ Consider a variation of the university organization where an instructor may be associated with *multiple departments*.

$inst (ID, dept_name, name, street, city)$

- ▶ Further assume that an instructor may have *several addresses*:

$ID \rightarrow name, ID \twoheadrightarrow street, city$

- ▶ Decomposition:

$ID \rightarrow name$ implies $ID \twoheadrightarrow name$

$r_1 (ID, name)$

$r_2 (ID, dept_name, street, city)$ (Not in 4NF)

$ID \twoheadrightarrow street, city$

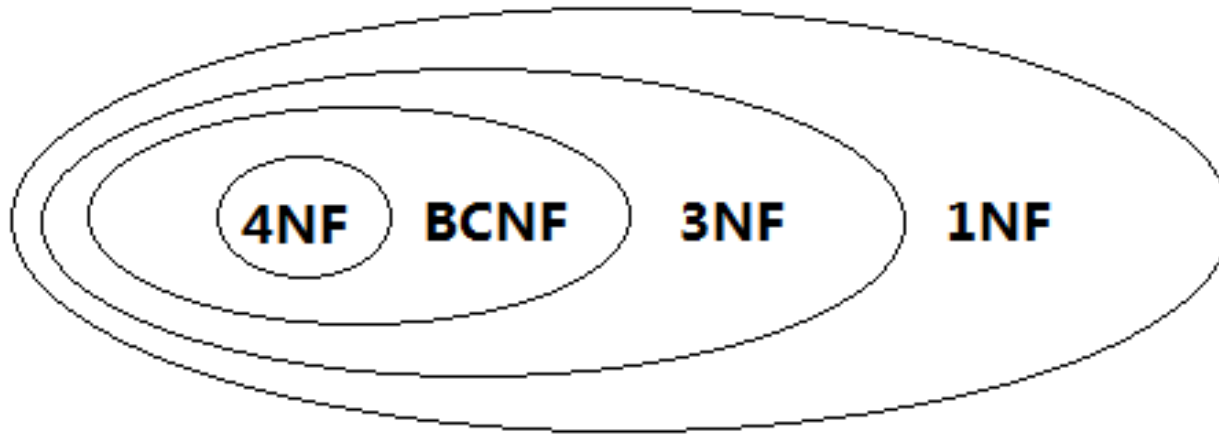
$r_{21}(ID, street, city)$

$r_{22}(ID, dept_name)$

▶ $(\alpha \cup \beta)$
▶ $(R - (\beta - \alpha))$



Review on Normal Forms



We have not discussed 2NF, because it is of historical interest only.



Further Normal Forms

- ▶ **Join dependencies** generalize multivalued dependencies.
 - ▶ lead to *project-join normal form (PJNF)*.
 - ▶ PJNF is also called **fifth normal form** in some books.
- ▶ A class of even more general constraints, leads to a normal form called *domain-key normal form (DKNF)*.
- ▶ **Problems** with these generalized constraints
 - ▶ They are hard to reason with, and no set of sound and complete set of inference rules exists.
 - ▶ Hence PJNF and DKNF are rarely used.



Database-Design Process



Overall Database Design Process

- ▶ We have assumed schema R is given. There are several ways to come up with the schema.
 - ▶ R could have been generated when *converting E-R diagram to a set of tables*.
 - ▶ R could have been a single relation containing *all* attributes that are of interest (called **universal relation**).
Normalization breaks R into smaller relations.
 - ▶ R could have been the result of some *ad hoc design of relations*, which we *then test/convert to normal form*.



E-R Model and Normalization

- ▶ When an **E-R diagram is carefully designed**, identifying all entities correctly, the tables generated from the E-R diagram *should not need further normalization*.
- ▶ However, in a real (imperfect) design, there can be functional dependencies *from non-key attributes of an entity to other attributes of the entity*
 - ▶ Example: an *employee* entity with attributes *department_name* and *building*, and a functional dependency *department_name* → *building*
 - ▶ Good design would have *made department an entity*.
- ▶ A relationship set involving more than two entity sets may result in a schema that may not be in a desirable normal form, but rare — *most relationships are binary*.



Naming of Attributes and Relationships

- ▶ A desirable feature of a database design is the ***unique-role assumption***.
- ▶ Each attribute name has a ***unique meaning*** in the database
 - ▶ Consider using the attribute ***number*** for phone number in the *instructor* schema and for room number in the *classroom* schema.
 - ▶ The ***join*** is meaningless. Having a different attribute name for each serves to reduce user errors.
- ▶ It may be a good idea to use the same attribute name if attributes of different relations have the ***same meaning***.
 - ▶ Considering the same attribute name “*name*” for both the *instructor* and the *student* entity sets.
- ▶ Relationship sets are often named via a ***concatenation of the names*** of related entity sets. But we cannot always create relationship-set names by simple concatenation.



Denormalization for Performance

- ▶ May want to use *non-normalized schema for performance*.
- ▶ For example, displaying *prereqs* along with *course_id*, and *title* requires join of *course* with *prereq*.
 - ▶ *Alternative 1*: Use *denormalized relation* containing attributes of *course* as well as *prereq* with all above attributes
 - ▶ *faster lookup*
 - ▶ *extra space and extra execution time* for updates
 - ▶ *extra coding work* for programmer and possibility of error in *extra code*
 - ▶ *Alternative 2*: Use a *materialized view* defined as $course \bowtie prereq$
 - ▶ Benefits and drawbacks same as above, except no extra coding work for programmer and avoids possible errors



Other Design Issues

- ▶ Some aspects of database design are not caught by normalization: *data pertaining to time or to ranges of time have several such issues.*
- ▶ *earnings (company_id, year, amount)*
 - ▶ *earnings_2004, earnings_2005, earnings_2006*, etc., all on the schema (*company_id, earnings*)
 - ▶ BCNF; but make querying across years difficult and needs new table each year
 - ▶ *company_year (company_id, earnings_2004, earnings_2005, earnings_2006)*
 - ▶ BCNF, but also makes querying across years difficult and requires new attribute each year.
 - ▶ an example of a **crosstab**, where values for one attribute become column names.
 - ▶ used in spreadsheets, and in data analysis tools.



Homework

► 7.30 注意写出推导步骤

7.30 Consider the following set F of functional dependencies on the relation schema (A, B, C, D, E, G) :

$$A \rightarrow BCD$$

$$BC \rightarrow DE$$

$$B \rightarrow D$$

$$D \rightarrow A$$

- Compute B^+ .
- Prove (using Armstrong's axioms) that AG is a superkey.
- Compute a canonical cover for this set of functional dependencies F ; give each step of your derivation with an explanation.
- Give a 3NF decomposition of the given schema based on a canonical cover.
- Give a BCNF decomposition of the given schema using the original set F of functional dependencies.

